

Android 第四章第二节教学讲义（小白入门版）

各位同学，今天咱们继续学习 Android 中 Activity 的进阶知识。上节课咱们认识了 Activity 的基本用法和生命周期，这节课咱们来聊聊 Activity 的启动模式、Intent 的两种类型，以及 Activity 之间的数据传递——这些都是实际开发中天天会用到的技能，咱们用大白话 + 代码例子来讲，保证你一听就懂！

一、Activity 启动模式：4 种 "打开页面" 的规则

咱们平时打开 APP 的页面时，可能会遇到这样的情况：比如打开微信聊天页，不管点多少次，每次都是新的聊天窗口；但打开设置页，可能每次都是同一个窗口。这就是因为 Activity 有不同的 "启动模式"，决定了新页面是新建一个、还是复用已有的。

Android 里有 4 种启动模式，就像 4 种 "打开页面的规则"，咱们一个个来看：

1. standard（标准模式）：默认规则，每次都新建

这是最常用的默认模式，就像 "每次打开都新建一个副本"。比如打开微信的 "添加好友" 页面，每点一次 "添加"，都会新打开一个一模一样的页面，点返回时要一个个退回去。

原理：每次启动 Activity，系统都会创建一个新的实例，不管之前有没有存在过。

代码设置：在 AndroidManifest.xml 里给 Activity 添加属性（不设置就是默认 standard）：

```
<activity
    android:name=".DetailActivity"
    android:launchMode="standard" /> <!-- 标准模式 -->
```

2. singleTop（栈顶复用）：栈顶相同则复用，否则新建

想象一下 "消息通知栏"：当你在消息列表页（栈顶），再次点击通知打开消息列表时，不会新建页面，而是复用当前的。但如果当前页面不是消息列表（不在栈顶），就会新建。

原理：如果要启动的 Activity 已经在 "任务栈顶部"（就是当前显示的页面），就直接复用，不会新建；如果不在栈顶，就新建一个。

适合场景：接收通知、刷新当前页面等。

代码设置：

```
<activity  
    android:name=".MessageActivity"  
    android:launchMode="singleTop" /> <!-- 栈顶复用 -->
```

3. singleTask（栈内唯一）：整个栈里只留一个，其他的"干掉"

比如手机的"设置"页面：不管你从哪个页面进入设置，整个任务栈里只会有一个设置页面，之前打开的其他设置相关页面都会被销毁。

原理：整个任务栈里最多只有一个该 Activity 的实例。如果已经存在，就把它上面的所有 Activity 都销毁，让它成为栈顶；如果不存在，就新建。

适合场景：首页、设置页等需要唯一实例的页面。

代码设置：

```
<activity  
    android:name=".HomeActivity"  
    android:launchMode="singleTask" /> <!-- 栈内唯一 -->
```

4. singleInstance（全局唯一）：单独占一个任务栈，整个系统只有一个

这个模式比较特殊，比如手机的"电话"页面：不管从哪个 APP 打开电话，都是同一个电话页面，而且它单独占一个任务栈，不和其他页面混在一起。

原理：整个系统中只有一个该 Activity 的实例，并且它独自占用一个任务栈，其他 APP 启动它时，直接复用这个实例。

适合场景：系统级页面（电话、闹钟）等。

代码设置：

```
<activity  
    android:name=".PhoneActivity"  
    android:launchMode="singleInstance" /> <!-- 全局唯一 -->
```

二、显示 Intent 和隐式 Intent：两种 "页面跳转的方式"

上节课咱们学了用 Intent 启动 Activity，其实 Intent 分两种：显示和隐式。就像 "寄快递"：显示 Intent 是 "直接写清楚收件人地址"，隐式 Intent 是 "写清楚收件人特征（比如 '北京市朝阳区 30 岁女性'）"，系统会根据特征找对应的收件人。

1. 显示 Intent：直接指定目标页面（常用）

最直接的跳转方式，明确告诉系统要启动哪个 Activity，就像 "快递直接写死收件人姓名和地址"。

用法：通过目标页面.class直接指定，上节课学的就是这种：

```
// 从MainActivity跳转到DetailActivity（明确指定目标）
Intent intent = new Intent(MainActivity.this, DetailActivity.class);
startActivity(intent);
```

特点：简单直接，适合自己 APP 内部的页面跳转。

2. 隐式 Intent：通过 "特征" 找目标页面（跨 APP 常用）

不直接指定目标 Activity，而是通过设置 "动作 (Action)"、"类别 (Category)"、"数据 (Data)" 等特征，让系统去匹配符合这些特征的 Activity。就像 "寄快递只写 '收：能打印文件的店'，系统会找附近的打印店"。

用法：需要在 AndroidManifest.xml 中给目标 Activity 设置 "特征标签"，然后在 Intent 中设置对应的特征。

步骤 1：给目标 Activity 设置特征（在 Manifest 中）

```
<activity android:name=".ShareActivity">
  <!-- 设置特征：能接收"分享"动作 -->
  <intent-filter>
    <action android:name="com.example.ACTION_SHARE" /> <!-- 自定义动作：分享 -->
    <category android:name="android.intent.category.DEFAULT" /> <!-- 默认类别（必须加） -->
  </intent-filter>
</activity>
```

```
</intent-filter>
</activity>
```

步骤 2：在源页面用隐式 Intent 匹配

```
// 创建隐式Intent，设置要匹配的动作
Intent intent = new Intent();
intent.setAction("com.example.ACTION_SHARE"); // 和目标Activity的action一致
startActivity(intent); // 系统会找到所有匹配该action的Activity，让用户选择（如果多个）
```

实际场景：调用系统功能（比如打开浏览器、发邮件）。例如打开百度网页：

```
// 隐式Intent调用系统浏览器（系统浏览器的特征是"浏览网页"）
Intent intent = new Intent(Intent.ACTION_VIEW); // 系统内置动作：查看
intent.setData(Uri.parse("https://www.baidu.com")); // 数据：网页地址
startActivity(intent);
```

三、Activity 之间的数据传递：互发 "小纸条"

咱们用 APP 时，经常需要传递数据：比如从 "商品列表页" 点击商品，把 "商品 ID" 传到 "商品详情页"；或者从 "选择头像页" 选完头像，把 "头像路径" 返回给 "个人资料页"。这就需要 Activity 之间传递数据。

1. 向下一个 Activity 发送数据（向前传）

比如从 MainActivity（首页）跳转到 DetailActivity（详情页），并传递一个字符串（如 "Hello Detail"）。

步骤 1：在源页面（MainActivity）中用 Intent 携带数据

Intent 就像一个 "小纸条"，可以用 `putExtra()` 方法往里面塞数据（键值对形式，比如 "key" 是名字，"value" 是内容）：

```
// 1. 创建Intent
Intent intent = new Intent(MainActivity.this, DetailActivity.class);
```

```
// 2. 往Intent里塞数据（键："message"，值："Hello Detail"）
intent.putExtra("message", "Hello Detail");
intent.putExtra("number", 100); // 还能传数字、布尔值等
// 3. 启动目标Activity，同时把数据带过去
startActivity(intent);
```

步骤 2：在目标页面（DetailActivity）中取出数据

在目标 Activity 的 `onCreate()` 方法中，用 `getIntent()` 拿到传递过来的 Intent，再用 `getStringExtra()` 等方法根据 "键" 取出数据：

```
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_detail);

    // 1. 获取传递过来的Intent
    Intent intent = getIntent();
    // 2. 根据键取出数据（第二个参数是默认值，防止没拿到数据时出错）
    String message = intent.getStringExtra("message");
    int number = intent.getIntExtra("number", 0);

    // 3. 用数据做事情（比如显示在文本框）
    TextView textView = findViewById(R.id.text_view);
    textView.setText(message + "，数字是：" + number);
}
```

2. 向上一个 Activity 返回数据（向后传）

比如从 "选择头像页"（SecondActivity）选完头像后，把 "头像路径" 返回给 "个人资料页"（FirstActivity）。这时候需要用 `startActivityForResult()` 启动，而不是普通的 `startActivity()`。

步骤 1：源页面（FirstActivity）启动目标页面时，说明 "要返回数据"

用 `startActivityForResult()` 启动，并指定一个 "请求码"（比如 100，用来区分不同的请求）：

```
// 1. 创建Intent
Intent intent = new Intent(FirstActivity.this, SecondActivity.class);
// 2. 启动目标页面，并说明"我要等你返回数据"（请求码100）
startActivityForResult(intent, 100);
```

步骤 2：在目标页面（SecondActivity）中设置返回数据

当用户完成操作（比如点击 "确定" 按钮），把要返回的数据塞到 Intent 里，用 `setResult()` 告诉系统 "我要返回数据了"，然后关闭当前页面：

```
// 假设用户点击了"确定"按钮，触发这个方法
public void onConfirmClick(View view) {
    // 1. 创建Intent，用来存返回的数据
    Intent returnIntent = new Intent();
    // 2. 塞数据（比如选中的头像路径）
    returnIntent.putExtra("avatar_path", "/sdcard/avatar.jpg");
    // 3. 告诉系统：返回成功，数据是returnIntent（RESULT_OK表示成功）
    setResult(RESULT_OK, returnIntent);
    // 4. 关闭当前页面，返回上一页
    finish();
}
```

步骤 3：源页面（FirstActivity）接收返回的数据

在源页面中重写 `onActivityResult()` 方法，这里会收到返回的数据（通过 "请求码" 确认是哪个请求，通过 "结果码" 确认是否成功）：

```
// 当目标页面返回数据时，系统会自动调用这个方法
@Override
protected void onActivityResult(int requestCode, int resultCode, Intent data) {
    super.onActivityResult(requestCode, resultCode, data);

    // 1. 先判断请求码是否是我们之前设置的100（确认是哪个请求）
    if (requestCode == 100) {
        // 2. 判断结果是否成功（RESULT_OK表示用户正常操作完成）
        if (resultCode == RESULT_OK) {
            // 3. 取出返回的数据
            String avatarPath = data.getStringExtra("avatar_path");
```

```
// 4. 用数据做事情（比如显示头像）
ImageView avatarImg = findViewById(R.id.avatar_img);
avatarImg.setImageURI(Uri.parse(avatarPath));
    }
}
}
```

总结

这节课咱们学了三个核心知识点，都是开发中必须掌握的：

1. **Activity 启动模式**：4 种规则决定页面是新建还是复用 ——standard（每次新建）、singleTop（栈顶复用）、singleTask（栈内唯一）、singleInstance（全局唯一）。
2. **Intent 的两种类型**：显示 Intent（直接指定目标，适合内部跳转）和隐式 Intent（通过特征匹配，适合跨 APP 跳转）。
3. **数据传递**：
 - 向前传：用 Intent 的putExtra()塞数据，目标页面用getIntent()取。
 - 向后传：源页面用startActivityForResult()启动，目标页面用setResult()返回，源页面重写onActivityResult()接收。

这些内容看起来多，但只要多写两个小例子（比如做一个 "列表页→详情页→返回列表页带数据" 的小功能），很快就能掌握。有问题随时问，咱们下节课见！

（注：文档部分内容可能由 AI 生成）